

An Interactive Sudoku System Driven by Constraint Satisfaction and Heuristic Backtracking

Md Al Amin

Department of Electrical and Computer
Engineering

North South University

al.amin25@northsouth.edu

Nuzhat Tasnim Silvia

Department of Electrical and Computer
Engineering

North South University

nuzhat.silvia@northsouth.edu

Saadia Rahman Oyshi

Department of Electrical and Computer
Engineering

North South University

saadia.oyshi@northsouth.edu

Sunbib Ahmed

Department of Electrical and Computer
Engineering

North South University

sunbib.ahmed@northsouth.edu

Abstract— The row, column, and block restrictions of Sudoku, a popular finite-domain constraint satisfaction problem (CSP), directly correspond to the all-different constraint. In this paper, a complete, playable 9×9 Sudoku application is presented. The same CSP solver that creates the puzzle also verifies the game's uniqueness, verifies player moves, and gives suggestions. Variables with domains $\{1, \dots, 9\}$ are used to handle empty cells. Search uses chronological backtracking ordered by the minimum-remaining-values (MRV) heuristic, so the engine expands the most constrained cell first. In order to create puzzles, a valid grid must be completed, structure-preserving shuffles must be applied, and clues must only be removed when the remaining instance still has a single solution. Three difficulty levels, a countdown timer, instantaneous correct/incorrect feedback, undo, a small hint channel, and an open scoring rule are all visible through a Tkinter interface. Four solvers are compared in controlled studies on thirty produced puzzles. The deployed MRV engine solves the same boards in 2.1 ms against 23.6 ms on Hard instances (about 26 clues), visiting a median of 83 nodes compared to 13,165 for naïve backtracking—almost a $160\times$ reduction. Adding the least-constraining-value (LCV) heuristic trims a few nodes on Hard boards but increases wall-clock time, so it is retained only as an analysis baseline. Therefore, rather than being a disconnected solver demo, the system shows how classical CSP approaches can power an interactive instructional game.

Index Terms— Constraint satisfaction, backtracking, minimum remaining values, Sudoku, puzzle generation, heuristic search, Tkinter

I. INTRODUCTION

A. Motivation

Constraint satisfaction is a fundamental subject in undergraduate artificial intelligence as it clearly delineates the three components of numerous combinatorial problems: variables, domains, and constraints [1], [2]. Sudoku illustrates a remarkably pure classroom representation of that approach. Every empty cell must be assigned a numeral from 1 to 9, and no numeral may be

duplicated within a row, a column, or a 3×3 grid. The regulations are concise enough to be articulated in one paragraph, yet achieving a uniform assignment is already NP-complete for generalized Sudoku [8]. The combined effect of clear regulations and complex search renders the puzzle an ideal medium for executing backtracking, domain filtering, and variable-ordering algorithms in user-friendly software.

The majority of student Sudoku applications either embed a limited number of boards or provide a solver devoid of any accompanying gameplay. The objective of this project's design is fundamentally different: all user-visible actions should be based on a singular constraint engine. The processes of generation, move checking, uniqueness testing, and tip text are not four separate scripts; rather, they constitute four calls within a singular CSP formulation.

B. Objectives

The established system was designed to fulfill the subsequent specific goals:

- Represent a conventional 9×9 Sudoku as a Constraint Satisfaction Problem and resolve it using backtracking with Minimum Remaining Values variable selection.
- Create a new problem for every game while favoring a singular answer.
- Deliver a comprehensive graphical game featuring locked hints, a keyboard interface, on-screen input, an undo function, a timer, error tracking, and scoring.
- Provide a clarifying suggestion that enumerates permissible options for a single cell and proposes a value without completing the entire board.
- Evaluate the performance of MRV in relation to naïve backtracking, forward checking, and the combination of MRV with LCV on identical created instances.

C. Contributions

Relative to a solver-only exercise, the working system contributes three artifacts. Initially, a generation pipeline that populates diagonal blocks, finalizes the grid with the CSP solver, rearranges bands and stacks, and eliminates hints based on a uniqueness criterion. Secondly, a closed loop wherein the hidden solution is employed for immediate visual reinforcement and secure suggestion guidance.

Third, a replicable standard (random seed 440, ten puzzles per difficulty, 21 timed iterations) clarifies the rationale behind the utilization of MRV as the search strategy in the released game: it surpasses naive search on Hard boards and outpaces a forward-checking variant, whose additional domain processing is not justified at this scale.

D. Organization

Section II reviews CSPs and Sudoku solvers. Section III delineates the formal framework. Section IV clarifies the framework, complexity, and evaluation criteria. Section V presents the algorithms. Section VI clarifies the execution. Section VII clarifies the experiments conducted.

Sections VIII and IX address constraints and provide a conclusion.

II. BACKGROUND AND RELATED WORK

A. Constraint Satisfaction Problems

A finite Constraint Satisfaction Problem (CSP) is defined as a triplet (X, D, C) [1], [2]. X constitutes a collection of variables, D allocates a limited domain to each variable, and C comprises a set of constraints that prohibit certain tuples of values. A solution is a comprehensive assignment that fulfills all requirements.

Chronological backtracking allocates a variable, performs recursion, and retracts the assignment upon failure [3]. The sequence in which variables are selected significantly influences the magnitude of the search tree. The MRV heuristic identifies the variable with the least current domain, implementing a "fail-first" strategy that uncovers dead-ends sooner [4], [5].

Subsequent to an assignment, forward checking eliminates the assigned value from the domains of neighboring unassigned variables and initiates backtracking if any domain is rendered empty [4]. The least constraining-value (LCV) heuristic prioritizes values that preserve greater options for adjacent variables.

B. Sudoku as a Constraint Satisfaction Problem

Simonis established that Sudoku forms a compact, organized CSP characterized by all-different predicates applied to rows, columns, and blocks [6]. In a 9×9 grid, the model contains a maximum of 81 variables and 27 constraints enforcing distinctness. Due to the regularity of the constraint graph and the minimal size of the domains, even a basic solver operates swiftly for interactive applications; the educational significance resides in observing how

a single-line heuristic modification (MRV compared to row-major order) impacts the tree.

Norvig's extensively circulated article delineates a Python Sudoku solution that integrates constraint propagation with search methodologies [7]. Commercial and open-source generators generally commence with a finalized Latin square-like grid and provide clues while ensuring originality [9]. The current initiative adheres to that framework but consolidates the generator, the in-game verifier, and the hint module under a unified implementation of domains and backtracking, instead of depending on an external SAT or MILP solver.

C. Interactive Tutoring Systems

The act of hinting in puzzle games constitutes a challenge in human-computer interaction as well as a search-related issue. A hint that dumps the full solution removes the exercise; a hint that only restates the rules may not help a stuck player. Candidate listing plus a single recommended digit is a compromise used here: the player still types the number, and the accompanying sentence names the row, column, and block constraints that keep the digit legal.

III. PROBLEM FORMULATION

Let P be a 9×9 matrix with $P_{r,c} \in \{0, 1, \dots, 9\}$, where 0 denotes an empty cell. The corresponding CSP is defined as follows

a) *Variables.*:

$$X = \{x_{r,c} \mid 1 \leq r, c \leq 9, P_{r,c} = 0\}. \quad (1)$$

b) *Domains.*: Each unassigned cell has the full digit set until constraints prune it:

$$D(x_{r,c}) = \{1, 2, \dots, 9\}. \quad (2)$$

A value v remains in the current domain if and only if it is absent from row r , column c , and the unique 3×3 block containing (r, c) . Writing $B(r, c)$ for that block,

$$\begin{aligned} \text{Legal}(P, r, c, v) \iff & v \notin \{P_{r,j}\}_{j=1}^9 \\ & \wedge v \notin \{P_{i,c}\}_{i=1}^9 \\ & \wedge v \notin \{P_{i,j}\}_{(i,j) \in B(r,c)}. \end{aligned}$$

The live domain used by search and by the hint module is therefore

$$Dp(x_{r,c}) = \{v \in \{1, \dots, 9\} \mid \text{Legal}(P, r, c, v)\}.$$

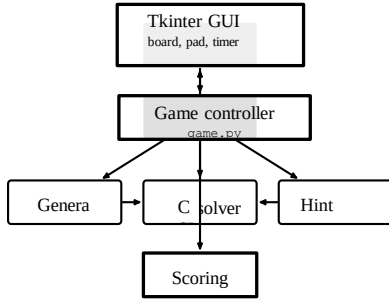


Fig. 1. Runtime architecture. The controller is the only module that talks to the player. Generator, hints, and move checking all depend on the CSP solver.

- c) Constraints.: For every row r , column c , and block b ,
all different $(\{x_r, 1, \dots, x_r, 9\})$ (5)

and likewise, for columns and blocks (given cells participate as already assigned values). This is exactly the standard Sudoku rule set.

- d) Search Decision.: MRV chooses the next variable according to

$$(r^*, c^*) = \arg \min_{\substack{(r,c) \\ |P_{r,c}|=0}} |DP(x_{r,c})|,$$

Dissolving connections by row-major scanning. A cell possessing an unoccupied domain is promptly returned to facilitate the caller's backtracking.

A comprehensive assignment that fulfills (5) constitutes a resolution. The generator also necessitates uniqueness: if $N(P)$ represents the quantity of solutions for P , a released puzzle must fulfill $N(P) = 1$.

In a standard Easy instance, there exist 41 holes. If MRV encounters a domain of size one, it populates that solitary hidden value prior to addressing a highly confusing cell. Row-major backtracking has no such inclination: it might allocate a six-candidate cell in the initial row and then realize that a size-one cell has been vacated. Equation (6) is hence not a method for optimizing the GUI; The reason that generation, verification, and suggestion remain within a few milliseconds, even at the elevated clue count.

IV. SYSTEM DESIGN AND METHODOLOGY

A. Overall Architecture

Figure 1 summarizes the functioning system. The Tkinter layer does not itself enforce the rules of Sudoku. It invokes the generator at the commencement of a game, retains the concealed solution, inquires of the solver if an entered number corresponds with that solution, and requests the hint module for the domain of

a specific cell. The score is only determined by the level of difficulty, the time left, the number of errors, and the quantity of hints utilized.

B. Gameplay Workflow

Figure 2 represents the loop visible to the player. The level of difficulty determines both the target clue and the countdown allocation (Table I). The timer remains inactive until the initial genuine action (selecting a cell, entering a digit, erasing, undoing, or requesting a clue), allowing a player to examine the board without consequence. Original hints remain unchanged. A matching digit is colored green; a mismatch is colored red and increases the error count. The session does not conclude upon errors; rather, every second mistake (at counts 2, 4, and 6) can activate an automatic hint, with a maximum of three tips available. Completing the board against the hidden solution opens a summary dialog. Expiration at 00:00 eliminates the existing grid and initiates a new distinctive puzzle at the identical level.

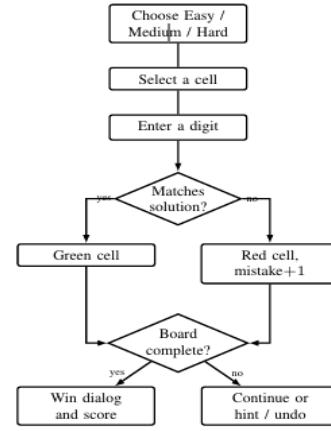


Fig.2. Player workflow implemented in game.py. Branching on correctness uses the hidden CSP solution, not a separate rule engine.

TABLE I
DIFFICULTY PARAMETERS USED BY THE RUNNING GAME

Level	Clues (target)	Time	Base score
Easy	40	10 min	1000
Medium	32	7 min	2000
Hard	26	5 min	3000

C. Difficulty Model

Clue counts and timers are consolidated into a single dictionary, allowing modifications without altering the search algorithm. Higher difficulty levels offer less hints and shorter time limits, while elevating the scoring threshold (Section IV-D)

D. Scoring Model

Let d be the chosen level, m the mistake count, h the number of hints consumed, and t the leftover seconds. During play the header shows only a level-and-penalty score so that the countdown does not appear to “eat” points every second:

$$S_{\text{play}}(d, m, h) = B(d) - 50m - 75h, \quad (7)$$

with $B(\text{Easy}) = 1000$, $B(\text{Medium}) = 2000$, $B(\text{Hard}) = 3000$. On a verified win the engine adds a time bonus and a completion bonus,

TABLE II
WORKED VALUES OF (8) FOR COMPLETED PUZZLES

Case	m	h	t (s)	S_{win}
Easy, clean finish	0	0	600	1850
Easy, two errors, one hint	2	1	400	1475
Hard, unused time	0	0	200	3850
Hard, quota used, $t = 0$	4	3	0	2825

where $\beta(\text{Easy}) = 1$, $\beta(\text{Medium}) = 2$, and $\beta(\text{Hard}) = 3$ points per leftover second. Table II instantiates (8) for four reachable end states. The rule rewards harder boards, independent solving (few hints), accuracy, and leftover time, without punishing the player for merely watching the clock.

V. ALGORITHMS

A. Constraint Check and Domain Construction

Listing1 is the Boolean test behind (3). It is deliberately side effect free: the same predicate is used while searching, while counting solutions, and while enumerating hint candidates. Building $D_p(xr, c)$ is a nine step loop over $=1, \dots, 9$.

```

1 def is_valid(board, row, col, number):
2     if number in board[row]:
3         return False
4     for r in range(9):
5         if board[r][col] == number:
6             return False
7     br, bc = 3 * (row // 3), 3 * (col // 3)
8     for r in range(br, br + 3):
9         for c in range(bc, bc + 3):
10            if board[r][c] == number:
11                return False
12     return True

```

Listing 1. Constraint test used by search, uniqueness, and hints (sudoku_solver.py).

B. MRV Reversal of Progression

Algorithm 1 identifies an MRV cell, evaluates each permissible contender, then reverts unsuccessful cells to zero using backtracking. It is complete and sound, and the first solution is unique by construction. For the purpose of uniqueness assessment, the search ceases upon discovering two solutions, hence establishing whether $N(P) \in \{1\}$ or $N(P) \geq 2$.

C. Puzzle Generation

Figure 3 illustrates the four-phase puzzle-creation procedure:

Diagonal sowing: Three diagonal sections are populated with arbitrary numerals ranging from 1 to 9. Algorithm 1 populates the unoccupied cells.

Rearranging: Rows, columns, bands, and numbers are altered without compromising validity [9]. **Unique carving:** Clues are randomly removed while ensuring the puzzle retains exactly one solution. The process stops when the difficulty target is reached.

If a New Game request provides the previous grid, generation is retried up to eight times to produce a different board.

Algorithm 1 SolveSudoku with MRV (project solver)

```

1: procedure SOLVE( $P$ )
2:    $(r, c) \leftarrow \text{MRVCELL}(P)$ 
3:   if  $(r, c)$  is nil then
4:     return true ▷ no empty cell
5:   end if
6:   for all  $v \in D_P(x_{r,c})$  do
7:      $P_{r,c} \leftarrow v$ 
8:     if SOLVE( $P$ ) then
9:       return true
10:    end if
11:     $P_{r,c} \leftarrow 0$  ▷ backtrack
12:  end for
13:  return false
14: end procedure

```

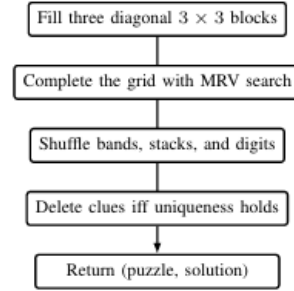


Fig. 3. Puzzle-generation pipeline implemented in sudoku_generator.py.

D. Hint Generation

Clues do not alter the grid. For a designated vacant cell, the system calculates $D_p(xr, c)$ and suggests the concealed solution value $S_{r,c}$ if it is still applicable. The explanation specifies the row, column, and section. A maximum of three clues is permitted.

```

1 def recommend_number(candidates, solution=None,
2                       row=None, col=None):
3     if not candidates:
4         return None
5
6     if solution is not None:
7         answer = solution[row][col]
8         if answer in candidates:
9             return answer
10    return candidates[0]

```

Listing 2. Hint recommendation prefers the unique solution digit (*ai_hint.py*).

TABLE III
DEVELOPMENT TOOLS AND SOURCE MODULES

Component	Role in the running system
Python 3 / Tkinter	Language and GUI toolkit
main.py	Process entry; loads the window
game.py	Controller: input, timer, undo, dialogs
sudoku_generator.py	Unique puzzle construction
sudoku_solver.py	CSP domains, MRV search, checking
ai_hint.py	Candidates, recommendation, text
scoring.py	Live and final score, eqs. (7)–(8)
performance_analysis.py	Four-solver benchmark (offline)

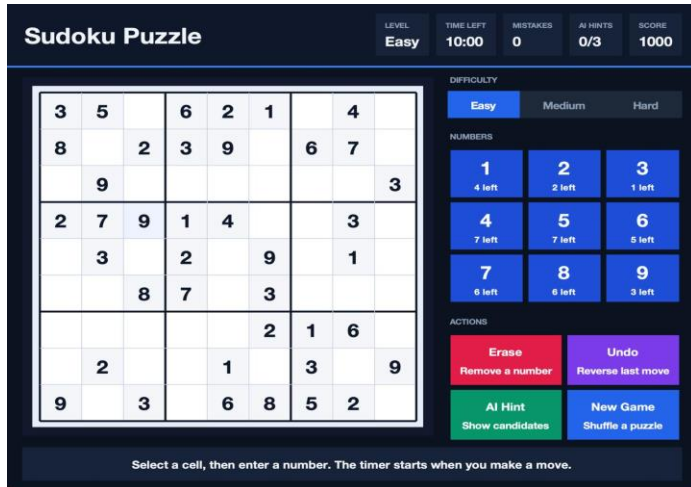


Fig. 4. Solved Easy board in the implemented interface. Green cells are player (or solver assisted) entries; gray cells are original clues. The digit pad reports every number as complete.

VI. Implementation

A. Tools and Modules

Table III encapsulates the software architecture. The game does not necessitate any external libraries; matplotlib is solely employed for benchmarking purposes.

B. Graphical Interface

The dark-themed interface includes game statistics, a Sudoku board, digit pad, and Erase, Undo, AI Hint, and New Game buttons. It emphasizes the chosen cell, associated cells, and

corresponding numerals. Figures 4 and 5 illustrate a finished puzzle along with its outcome dialog.

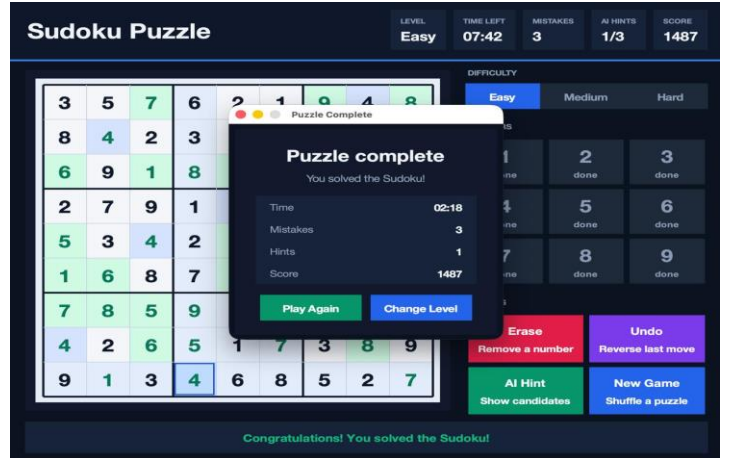


Fig. 5. Completion dialog after a correct Easy solve. The illustrated auto-solve finished with zero mistakes and zero hints; leftover time therefore remains 10:00 and the score is 1850 by (8).

C. Input, Undo, and Safety

Players can enter digits using the keyboard or digit pad and navigate with arrow keys. Erase and Undo exclusively influence cells occupied by players. Incorrect entries are ignored during hint-domain calculations.

VII. Experimental Evaluation

A. Protocol

The benchmark generated ten puzzles for each difficulty using seed 440. Four algorithms were tested:

Naive backtracking

Backtracking with forward checking

CSP + MRV

CSP + MRV + LCV

Each puzzle was timed over 21 runs using the median result. Nodes, assignments, and backtracks were also recorded. All 120 algorithm–puzzle pairs produced the correct solution.

B. Search-Tree Size

Figure 6 and Table V present the median nodes. On Easy boards, MRV attains the minimum threshold of 42 nodes. In Medium boards, naïve search expands to 732 nodes, whereas MRV remains constant at 50. In the context of Hard boards, naïve search traverses a median of 13,165 nodes, whereas MRV only visits 83, resulting in a decrease of around 159 times. Forward checking diminishes the Hard median to 3,351 nodes, although LCV marginally enhances MRV from 83 to 78.

TABLE IV
MEDIAN NODES VERSUS NUMBER OF EMPTY CELLS

Level	Empty	Naive	MRV	Naive/MRV
Easy	41	117	42	2.8×
Medium	49	732	50	14.6×
Hard	≈ 55	13,165	83	159×

TABLE V
MEDIAN NODES VISITED ($n = 10$ PUZZLES). GAME SOLVER IN BOLD.

Algorithm	Easy	Medium	Hard
Naive backtracking	117	732	13,165
BT + forward checking	57	262	3,351
CSP + MRV (game)	42	50	83
CSP + MRV + LCV	42	50	78

The outcomes of backtracking exhibit a consistent pattern. MRV

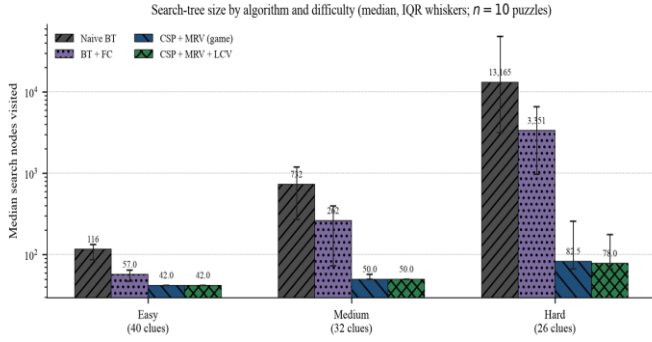


Fig. 6. Median search nodes (log scale) with 25th–75th percentile whiskers, ten puzzles per difficulty. The game solver is the third bar in each group.

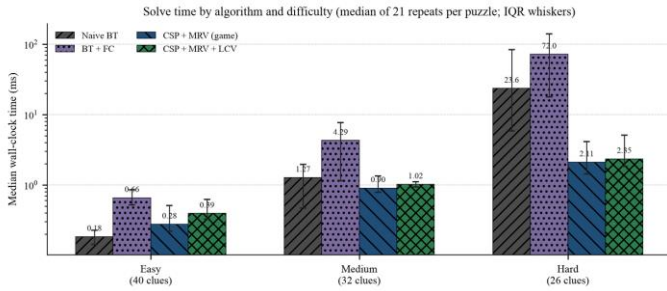


Fig. 7. Median solve time in milliseconds (log scale). Each bar is the median, over ten puzzles, of a 21-repeat median. Whiskers mark the interquartile range.

documents a median of 0 backtracks on Easy and Medium levels, and 27 on Hard, in contrast to 13,109 for a naive Hard search. Table IV indicates that the benefit of MRV escalates with the removal of additional hints.

C. Wall-Clock Time

Figure 7 and Table VI illustrate that MRV resolves Hard riddles in 2.11 ms, roughly $\backslash(11\backslash\times\backslash)$ swifter than naive search (23.57 ms) and 34times more rapid than forward checking (71.96 ms). Forward checking traverses a reduced number of nodes while incorporating expensive domain verifications. In a comparable manner, LCV diminishes the number of nodes marginally while extending the runtime to 2.35 ms. Consequently, the game use solely MRV.

D. Functional Behaviour

The application was additionally evaluated in relation to the GUI behaviors described in Table VII. Every examination signifies a realized and attainable user engagement.

TABLE VII
FUNCTIONAL CHECKS ON THE IMPLEMENTED GAME

Scenario	Observed behaviour
Locked clue	Typed digits and Erase are rejected
Correct digit	Cell turns green; score unchanged by time
Incorrect digit	Cell turns red; mistakes increment
Two mistakes	Automatic hint if quota remains
Hint on empty cell	Popup lists domain and one recommendation
Fourth hint attempt	Status: no hints remaining
Undo / Erase	Restores a player cell, not a clue
Timeout at 00:00	New unique puzzle, same level
Full correct board	Dialog with time, mistakes, hints, score

TABLE VI
MEDIAN SOLVE TIME IN MILLISECONDS. GAME SOLVER IN BOLD.

Algorithm	Easy	Medium	Hard
Naive backtracking	0.18	1.27	23.57
BT + forward checking	0.66	4.29	71.96
CSP + MRV (game)	0.28	0.90	2.11
CSP + MRV + LCV	0.39	1.02	2.35

As a numeric illustration of (8), an Easy board solved with $m=h=0$ and $t=600$ scores $1000+600\cdot1+250 = 1850$, matching Fig. 5.

E. Threats to Validity

Internal validity is strong because every solver uses the same 30 boards and unique solutions. External validity is limited because puzzles from other sources may have different structures. Timing results are based on C Python, one machine, and the median of 21 runs, so they indicate general performance rather than hardware rankings. Clue count is also only a proxy for human difficulty [6].

VIII. DISCUSSION

A. What the Measurements Support

The experimental assertion is both modest and precise: for the problems produced by this game, selecting the most constrained vacant cell distinguishes a search tree that remains close to the number of gaps from one that expands dramatically when hints enter the Hard category. This represents the traditional rationale for MRV [1], [4], reiterated here with the project's own generator instead of selectively chosen newspaper puzzles.

Forward checking continues to be a valid and frequently effective method [4], [5]. It is slower here not because the idea is wrong, but because 9×9 Sudoku domains are so small that an extra peer scan costs more than it saves until the board is much emptier than Easy or Medium. LCV is a value-ordering heuristic whose benefit is likewise instance-dependent; on this suite it is a regression in time and only a slight win in nodes, so including it in the interactive loop would complicate the code without helping the player.

B. Design Choices that Matter in Play

Employing the concealed resolution for green/red feedback serves as a practical substitute for only verifying local consistency. A locally legal digit can still be globally wrong on a unique puzzle; marking it red teaches the intended completion rather than a different valid grid that the generator already excluded. Hints adhere to the same principle: they advocate for Sr,c , never simply a legal substitute, so the assistance channel cannot oppose the verifier. alternative solution. The disclosed hint counts in the CSV are thus “approximately” 40/32/26. This is a genuine characteristic of uniqueness-preserving generation, rather than a flaw in the timing table.

C. Limitations:

The project does not purport to be a commercial Sudoku engine. There exists no dancing-links exact-cover algorithm [10], no AC-3 arc consistency extending beyond the implied domain of

(4), and no human-like pattern repository (naked pairs, X-wings, etc.). Difficulty serves as a proxy for clue count; it is not adjusted to align with a human assessment scale. The graphical user interface is exclusively for desktop use (Tkinter) and has not been assessed in a user survey. The standard employs thirty puzzles from a singular seeded generator; a more extensive corpus might enhance the precision of confidence intervals regarding the Heavy-tailed naïve Hard times. Finally, automatic hints after mistakes help beginners but can be turned off only by exhausting the quota—there is no separate “tutor mode” switch.

IX. CONCLUSION

This project successfully implements a complete and intelligent Sudoku game using a Constraint Satisfaction Problem (CSP) model with Minimum Remaining Values (MRV) backtracking. The identical computational engine underpins the primary functionalities of the application, such as producing legitimate and distinctive puzzles, verifying player inputs, counting possible responses, resolving incomplete boards, and offering one-cell hints accompanied by clear explanations. Utilizing a communal CSP framework, the system ensures coherence across problem creation, gameplay verification, hint production, and automated resolution.

The experimental assessment contrasted the project solver with naïve backtracking, backtracking enhanced by forward checking, and constraint satisfaction problems utilizing Minimum Remaining Values (MRV) and Least Constraining Value (LCV). The findings indicate that MRV offers the optimal equilibrium between search efficiency and execution duration for a standard 9×9 . Sudoku grid. In challenging puzzles, it decreases the median count of examined nodes by roughly two orders of magnitude and is consistently about one order of magnitude swifter than basic backtracking. Although forward checking and LCV can reduce parts of the search tree, their additional domain-checking and candidate-ordering operations introduce computational overhead. Consequently, the ultimate application employs MRV devoid of superfluous heuristics

The initiative possesses instructional significance as it renders a traditional artificial intelligence method observable via an interactive application. A learner has the ability to engage with the game, solicit a hint, examine the scoring mechanism, and analyze `sudoku_solver.py` to comprehend the interplay of legal-value verification, MRV-oriented cell selection, recursive backtracking, and solution enumeration. The main contribution of this CSE 440 project is not the invention of a new constraint-solving algorithm, but the practical integration of a well-established algorithm into puzzle generation, gameplay, tutoring, scoring, and measurable performance evaluation.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2021, ch. 6.
- [2] R. Dechter, *Constraint Processing*. San Francisco, CA, USA: Morgan Kaufmann, 2003.
- [3] J. R. Bitner and E. M. Reingold, "Backtrack programming techniques," *Commun. ACM*, vol. 18, no. 11, pp. 651–656, Nov. 1975.
- [4] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," *Artif. Intell.*, vol. 14, no. 3, pp. 263–313, 1980.
- [5] P. van Beek, "Backtracking search algorithms," in *Handbook of Constraint Programming*, F. Rossi, P. van Beek, and T. Walsh, Eds. Amsterdam, The Netherlands: Elsevier, 2006, pp. 85–134.
- [6] H. Simonis, "Sudoku as a constraint problem," in *Proc. 4th Int. Workshop Modelling and Reformulating Constraint Satisfaction Problems, 2005*, pp. 13–27.
- [7] P. Norvig, "Solving every Sudoku puzzle," 2006. [Online]. Available: <https://norvig.com/sudoku.html>
- [8] T. Yato and T. Seta, "Complexity and completeness of finding another solution and its application to puzzles," *IEICE Trans. Fundamentals*, vol. E86-A, no. 5, pp. 1052–1060, 2003. 8 CSE 440 FINAL REPORT
- [9] B. Felgenhauer and F. Jarvis, "Enumerating possible Sudoku grids," 2005. [Online]. Available: <http://www.afjarvis.staff.shef.ac.uk/sudoku/>
- [10] D. E. Knuth, "Dancing links," in *Millennial Perspectives in Computer Science*, J. Davies, B. Roscoe, and J. Woodcock, Eds. Basingstoke, U.K.: Palgrave, 2000, pp. 187–214.
- [11] Python Software Foundation, "Python 3 documentation." [Online]. Available: <https://docs.python.org/3/>
- [12] Python Software Foundation, "Graphical user interfaces with Tk (tkinter)." [Online]. Available: <https://docs.python.org/3/library/tkinter.html>
- [13] J. D. Hunter, "Matplotlib: A 2D graphics environment," *Comput. Sci. Eng.*, vol. 9, no. 3, pp. 90–95, 2007.
- [14] Cursor, "Cursor IDE documentation," AnySphere, 2026. [Online]. Available: <https://cursor.com>